# MATH 174 - Numerical Analysis

Dani Nguyen

Fall 2025

## Contents

# 1 Error

## 1.1 Approximation

Let x be an exact number.
Let y be an approximation of x.
**Def:** The absolute error is $|x - y|$.
**Def:** The relative error is $\frac{|x-y|}{|x|}$,

## 1.2 Round-off Error

Usually a machine uses a finite amount of memory to store a number.
machine $\leftarrow \pi$
Suppose a machine can store some number of digits.

1. Allocate a digit for sign, positive or negative.
2. Allocate a number of digits for the mantissa.
3. Allocate a digit for sign of the exponent.
4. Allocate a number of digits for the exponent.

Under rounding in the case of a 4 digit mantissa, $\pi = 0.3142 \cdot 10^1$, this is the machine # representation of pi.

**Def:** Round-off error are all errors arising from the error between actual # and machine #.

**Notation:**

$$x = \text{real \#}$$
$$fl(x) = \text{machine \# representation of } x$$

In an s-digit rounding machine,

Absolute error $= |x = fl(x)|$
Relative error $= \dfrac{|x - fl(x)|}{|x|} \leq \frac{1}{2} \cdot 10^{1-5} = 5 \cdot 10^{-5}$

## 1.3 Floating Point Arithmetic

What happens under arithmetic operations? (+, -, $\cdot$, /)

Two things to watch out for:

1. Many operations can degrade error.

2. Catastrophic cancellation.

### 1.3.1 Degrading Error

| number | machine |
|---|---|
| $x$ | $fl(x)$ |
| $y$ | $fl(y)$ |
| $x + y$ | $fl(fl(x) + fl(y))$ |

The more nested $fl()$ there are, the larger the error will be. The same follows for -, $\cdot$, /.

When performing arithmetic, it temporarily allows for more digits than the mantissa allows.

Algorithmic "solutions"
$((x + y) + z) + w$
Worst case, 4 times the error.
$(x + y) + (z + w)$
Worst case, 3 times the error.

### 1.3.2 Catastrophic Cancellation

**Ex:** 3-digit rounding machine. Given 2 numbers: 0.312 and 0.311.

$$0.312 - 0.311 = 0.001 = 0.1 \cdot 10^{-2}$$

Loss of significant digits!

**Ex:** 5-digit rounding machine. Given 2 numbers:

$$\begin{aligned} x &= 0.41626324 \\ y &= 0.41626323 \\ x - y &= 0.00000001 \\ \text{In machine:} & \\ fl(x) &= 0.41626 \\ fl(y) &= 0.41626 \\ fl(x) - fl(y) &= 0.00000 = 0 \\ fl(fl(x) - fl(y)) &= 0 \\ \text{Relative error:} & \\ \frac{|x - y - fl(fl(x) - fl(y))|}{|x - y|} &= \frac{|x - y - 0|}{|x - y|} \\ &= 1 = 100\% \text{ error!} \end{aligned}$$

# 2 Speed

## 2.1 Dot Product

Consider dot product of vectors

$$\vec{x} \in \mathbb{R}^n, \vec{y} \in \mathbb{R}^n$$

$$\vec{x} \cdot \vec{y} = \begin{bmatrix} x_1 & x_2 & \dots & x_n \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

$$= x_1 y_1 + x_2 y_2 + \ldots + x_n y_n$$

Count time-consuming operations:

1. Add/subtract
2. Multiply/divide
3. Comparisons
4. Memory access/storage

These are called FLOPS (floating point operations).

For the dot product, there are n multiplications and n-1 additions.
Adding these together, we get $2n-1$ FLOPS.
*Note that this expression is linear in n.*

For large $n$, 2n dominates, $= \mathcal{O}(n) \approx$ constant times $n$.
Ex: For $n$ vs $2n$, $2n$ will take twice as long.
Ex: For $\mathcal{O}(n^2)$, $n$ vs $2n$, $2n$ will take four times as long.

## 2.2 Memory

Consider a matrix $A_{n\times n}$

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} vs \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

The square matrix uses $n^2$ memory locations, while the vector uses $n$ memory locations. Each number also uses 8 bytes, creating another goal for algorithm development, reduce memory usage.

# 3 Gaussian Elimination

## 3.1 Triangular Matrices

Write a MATLAB function that when given $A\vec{x} = \vec{b}$, computes $\vec{x}$

$$A\vec{x} = \vec{b}$$

A is triangular and invertible

Let's say $A$ is lower triangular. $a_{ij} = 0, \forall j > i$

$$\begin{bmatrix} a_{11} & 0 & 0 & \dots & 0 \\ a_{21} & a_{22} & 0 & \dots & 0 \\ a_{31} & a_{32} & a_{33} & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}$$

Similarly, if $A$ is upper triangular, $a_{ij} = 0, \forall j < i$.

**Ex:** $A\vec{x} = \vec{b}$, A is upper triangular.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

This gives us the following system of equations:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{33}x_3 &= b_3 \end{aligned}$$

So $x_3 = \frac{b_3}{a_{33}}, a_{33} \neq 0$ if A is invertible.

Then, $x_2 = \frac{b_2 - a_{23}x_3}{a_{22}}, a_{22} \neq 0$ if A is invertible.

Finally, $x_1 = \frac{b_1 - a_{12}x_2 - a_{13}x_3}{a_{11}}, a_{11} \neq 0$ if A is invertible.

This is called back substitution.
For $i = n, n-1, \ldots, 1$

$$\begin{aligned} x_i &= \frac{b_i - (a_{i,i+1}x_{i+1} + \ldots + a_{i,n}x_n)}{a_{ii}} \\ &= \frac{b_i - \sum_{j=i+1}^{n} a_{ij}x_j}{a_{ii}} \end{aligned}$$

**Ex:** $A\vec{x} = \vec{b}$, A is lower triangular.

$$\begin{bmatrix} a_{11} & 0 & 0 \\ a_{21} & a_{22} & 0 \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

This gives us the following system of equations:

$$\begin{aligned} a_{11}x_1 &= b_1 \\ a_{21}x_1 + a_{22}x_2 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3 \end{aligned}$$

This is called forward substitution.
For $i = 1, 2, \ldots, n$

$$\begin{aligned} x_i &= \frac{b_i - (a_{i,1}x_1 + \ldots + a_{i,i-1}x_{i-1})}{a_{ii}} \\ &= \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j}{a_{ii}} \end{aligned}$$

Now given a regular matrix, make it triangular without changing the solutions. Recall Gaussian elimination.

1. $E_i$ (equation i) replaced by $cE_i$. Row scaling.
2. $E_i$ swapped with $E_j$. Row swapping.
3. $E_i$ replaced by $E_i + cE_j$. Row addition.

### 3.1.1 Speed of Back Substitution

For $i = n, n-1, \ldots, 1$

$$x_i = \frac{b_i - \sum_{j=i+1}^{n} u_{ij}x_j}{u_{ii}}$$

| | |
|---|---|
| $i = n$ | 1 flop |
| $n-1$ | 3 flops |
| $\vdots$ | $\vdots$ |
| 1 | $2n-1$ flops |

$$\sum_{j=1}^{n}(2j-1) = 2\sum_{j=1}^{n}(j) - \sum_{j-1}^{n} 1 = 2\left(\frac{n(n+1)}{2}\right) - n = n^2$$

With a large $n$, the term $n^2$ dominates, meaning $\mathcal{O}(n^2)$

## 3.2 Augmented Matrices

Gaussian Elimination on same matrices and different RHS comes up often in applications and algorithms.

### 3.2.1 LU Factorization

$$A = LU$$

where L is lower triangular with 1's in the diagonal and U is upper triangular

The important entries of L are called multipliers. These entries can be found with the negative of the coefficients of the steps of Gaussian elimination.

$$\begin{aligned} L\vec{y} &= \vec{b} \quad \text{get y from forward substitution} \\ U\vec{x} &= \vec{y} \quad \text{get x from back substitution} \end{aligned}$$

## 3.3 Matrix Inverses

Matrix-vector multiplication:

$$(A\vec{x})_i = \sum_{j=1}^{n} a_{ij}x_j$$

which gives $2(n-1)n$ flops total

Backwards substitution uses $n^2$ flops.
Forward substitution uses $n^2 - n$ flops. Because we're working with L, which has 1's on the diagonal, we save $n$ divisions.

In summary, getting $A^{-1}$ to solve $A\vec{x} = \vec{b}$ is slower. Also, $A^{-1}$ has memory storage issues for certain matrices. For example, a tridiagonal matrix, even though it has zeroes on most of its entries, $A^{-1}$ has no zeroes.

### 3.3.1 Gaussian Elimination FLOP count

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ 0 & a_{22} & a_{23} & \dots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}$$

This step has 1 division, 1 multiplication, 1 addition/subtraction.
In total there are $2n-1$ total flops.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ 0 & a_{22} & a_{23} & \dots & a_{2n} \\ 0 & a_{32} & a_{33} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}$$

Repeat this for $n-1$ rows (minus 1 because we are excluding the first row).
We can then repeat this for the inner block matrix.

$$(2n-1)(n-1) + (2(n-1)-1)((n-1)-1) + (2(n-2)-1)((n-2)-1)\dots$$

$$\sum_{j-1}^{n} j = \frac{n(n+1)}{2}$$

$$\sum_{j-1}^{n} j^2 = \frac{n(n+1)(2n+1)}{6}$$

# 4 Gaussian Elimination With Pivoting

But what about $A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ Gaussian elimination fails on first step. Can we use another elementary row operation, namely, row swapping?

In general,

$$\begin{bmatrix} 0 & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}$$

## 4.1 Error

Find a nonzero element $a_{i1}$ then perform $E_1 \leftrightarrow E_2$. We have to do this before each elimination step. However, there is error involved.

$$\left[\begin{array}{cc|c} \epsilon & 1 & 1 \\ \times & \times & \times \end{array}\right] \rightarrow \tilde{x}_2 = x_2 + \delta$$

$$\rightarrow \tilde{x}_1 = \frac{1 - \tilde{x}_2}{\epsilon} = \frac{1 - x_2 - \delta}{\epsilon} = \frac{1 - x_2}{\epsilon} - \frac{\delta}{\epsilon}$$

Note that in this situation, choosing an $x_1$ that is small leads to a small $\epsilon$ which leads to a large error. We have to be careful when swapping rows.

$$\left[\begin{array}{cc|c} \epsilon & 1 & 1 \\ 0 & \times & \times \end{array}\right]$$

$$\begin{aligned} \tilde{x}_2 &= x_2 + \delta \\ |\delta| &= |\tilde{x}_2 - x_2| \\ \tilde{x}_1 &= x_1 - \frac{\delta}{\epsilon} \\ \left|\frac{\delta}{\epsilon}\right| &= |\tilde{x}_1 - x_1| \end{aligned}$$

### 4.1.1 Section 7.5

$$A\vec{x} = \vec{b}$$
$$A(\vec{x} + \vec{\delta x}) = \vec{b} + \vec{\delta b}$$
$$A\vec{\delta x} = \vec{\delta b}$$
$$\vec{\delta x} = A^{-1}\vec{\delta b}$$

We can use a norm to measure the size of error vectors.

$$||\vec{\delta x}|| = ||A^{-1}\vec{\delta b}|| \leq ||A^{-1}||\ ||\vec{\delta b}||$$
$$\frac{||\vec{\delta x}||}{||\vec{x}||} \leq ||A||\ ||A^{-1}|| \frac{||\vec{\delta b}||}{||\vec{b}||}$$

relative error $\leq$ condition # $\times$ relative size of change of $\vec{b}$

**Ex:** Suppose you need to solve accurately $A\vec{x} = \vec{b}$ and someone gives you the approx $\tilde{x}$, is it accurate?

We can use the residual vector.

$$\vec{r} = \vec{b} - A\tilde{x}$$
$$A\tilde{x} = \vec{b} - \vec{r}$$

If A is well-conditioned (small condition number), then $\frac{||\vec{r}||}{||\vec{b}||}$ is small and $\frac{||\vec{x}-\tilde{x}||}{||\vec{x}||}$ small.

If A is ill-conditioned (large condition number), then we don't know.

## 4.2 Row Swapping

$$PA = LU$$

where PA is A with rows swapped. P is the identity matrix with its rows swapped to denote the row swapping operations done on A.

Now solving LU factorization:

$$\begin{aligned} A\vec{x} &= \vec{b} \\ PA\vec{x} &= P\vec{b} \\ LU\vec{x} &= P\vec{b} \\ L\vec{y} &= P\vec{b} \quad \text{forward sub} \\ U\vec{x} &= \vec{y} \quad\quad \text{back sub} \end{aligned}$$

### 4.3 Speed

In an $n \times n$ matrix, it takes $n-1$ comparisons for the first step, $n-2$ for the second step, and so on.

$$\begin{aligned} \sum_{j=1}^{n-1} j &= \frac{(n-1)n}{2} \\ &= \mathcal{O}(n^2) \end{aligned}$$

This is negligible additional amount of work for large $n$ compared to $\mathcal{O}(n^3)$ of Gaussian elimination.

Even safer than row partial pivoting in terms of error:

1. Scaled row partial pivoting:
2. Complete pivoting: row swaps and column swaps to move the largest element to pivot element.

## 5 Approximations

### 5.1 Fixed Point Problems

Solve an equation $g(x) = h(x)$
Modify to $f(x) = x$

Now we have the fixed point problem, which we can try to solve using fixed point iterations.

Start with guess $x^{(0)}$
Then iterate $x^{(k+1)} = f(x^{(k)})$
The $x^{(k)}$ are approximations of the fixed point.

We can also write $A\vec{x} = \vec{b}$ as fixed point problem using splitting.

$$\begin{aligned} A &= M - N \\ (M - N)\vec{x} &= \vec{b} \\ M\vec{x} &= N\vec{x} + \vec{b} \\ \vec{x} &= M^{-1}(N\vec{x} + \vec{b}) \end{aligned}$$

where the RHS is $f(\vec{x})$

Choices for M, N:

Let $A = D - L - U$, where D is diagonal, L is lower triangular, U is upper triangular. We have

1. Jacobi iterative method.
2. Gauss-Seidel iterative method.

### 5.1.1 Jacobi Iterative Method

$$\begin{aligned} M &= D \\ N &= L + U \\ D\vec{x} &= (L + U)\vec{x} + \vec{b} \end{aligned}$$

$$x_i^{(k+1)} = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)}}{a_{ii}}$$

### 5.1.2 Gauss-Seidel Iterative Method

$$\begin{aligned} M &= D - L \\ N &= U \\ (D - L)\vec{x} &= U\vec{x} + \vec{b} \end{aligned}$$

$$x_i^{(k+1)} = \frac{b_i - \sum_{j>i} a_{ij} x_j^{(k)} - \sum_{j<i} a_{ij} x_j^{(k+1)}}{a_{ii}}$$

## 5.2 Convergence

Jacobi & Gauss-Seidel are iterative methods. They start with a guess, which improves their approximations at each step. We want $\vec{x}^{(k)}$ sequence of approximations to converge to the exact solution.

Practically, we need a stopping condition. Here are our choices:

1. Stop after certain number of steps
2. Stop when residual is small, where the residual is $\vec{r} = \vec{b} - A\vec{x}$, $||\vec{r}|| \leq$ tolerance.
3. Stop when the algorithm slows down, $||x^{(k+1)} - x^{(k)}|| \leq$ tolerance.

$$\begin{aligned}
A &= M - N \\
\text{Jacobi} \quad M &= D \\
\text{Gauss-Seidel} \quad M &= D - L \\
A\vec{x} = \vec{b} &\rightarrow (M - N)\vec{x} = \vec{b} \\
&\rightarrow M\vec{x} = N\vec{x} + \vec{b} \\
&\rightarrow \vec{x} = M^{-1}(N\vec{x} + \vec{b}) \\
&\rightarrow \vec{x}^{(k+1)} = M^{-1}(N\vec{x}^{(k)} + \vec{b}) \\
&\rightarrow \vec{x}^{(k+1)} = \mathsf{T}\vec{x}^{(k)} + \vec{c} \\
&\text{where } \mathsf{T} \text{ is the iteration matrix, } M^{-1}N \\
&\text{and } \vec{c} \text{ is } M^{-1}\vec{b}
\end{aligned}$$

An iterative method using $\vec{x}^{(k+1)} = \mathsf{T}\vec{x}^{(k)} + \vec{c}$ converges if $\varphi(\mathsf{T}) < 1$, where $\varphi(\mathsf{T})$ is the spectral radius of $\mathsf{T}$.

**Def:** Spectral radius $\varphi(A) = \max\{|\lambda_1|, |\lambda_2|, \ldots, |\lambda_n|\}$

$$||\vec{e}^{\,k+1}|| \approx \varphi(T)||\vec{e}^{\,k}||$$
$$||\vec{e}^{\,k}|| \approx \varphi(T)^k||\vec{e}^{\,(0)}||$$

Let A = $\begin{bmatrix} a & b \\ b & c \end{bmatrix}$

$T_J = \begin{bmatrix} 0 & \frac{-b}{a} \\ \frac{-b}{c} & 0 \end{bmatrix}$

$T_{GS} = \begin{bmatrix} 0 & \frac{-b}{a} \\ 0 & \frac{b^2}{ac} \end{bmatrix}$

$\det T_J = \begin{bmatrix} -\lambda & \frac{-b}{a} \\ \frac{-b}{c} & -\lambda \end{bmatrix} = \lambda^2 - \frac{b^2}{ac}$

$\det T_{GS} = \begin{bmatrix} -\lambda & \frac{-b}{a} \\ 0 & \frac{b^2}{ac} - \lambda \end{bmatrix} = \lambda(\lambda - \frac{b^2}{ac})$

$\lambda_J = \pm\sqrt{\frac{b^2}{ac}} = \varphi(T_J)$

$\lambda_{GS} = 0$ or $\frac{b^2}{ac}$, $\varphi(T_{GS}) = \frac{b^2}{ac}$

If they converge, then $\frac{b^2}{|ac|} < 1$, and when this condition is true, Gauss-Seidel's method has a lower spectral radius than Jacobi, since $x < \sqrt{x}$ for x $< 1$.

However, there are examples for general matrices where Jacobi's method converges and Gauss-Seidel's method does not. So for general matrices, we do not know which one is faster. In practice, however, Gauss-Seidel is almost always faster.

Other iterative methods:

1. Richardson's
2. Successive Over-Relaxation (SOR) - Gauss-Seidel is in this class
3. Conjugate Gradient (not splitting)
4. GMRES

### 5.2.1 Richardson's Iterative Method

$$
\begin{aligned}
A\vec{x} &= \vec{b} \\
\omega A\vec{x} &= \omega\vec{b} \\
\omega A &= I - I + \omega A \\
&= I - (I - \omega A) \text{ where I is M, and } (I - \omega A) \text{ is N}
\end{aligned}
$$

$$
\begin{aligned}
x^{k+1} &= M^{-1}(Nx^k + \omega b) \\
&= Nx^k + \omega b \\
&= (I - \omega A)x^k + \omega b \\
x_i^{k+1} &= x_i^k - \omega \sum_{j=1}^{n} a_{ij}x_j^k + \omega b_i
\end{aligned}
$$

# 6 Eigenvalues

## 6.1 Finding Eigenvalues of a Matrix

Given $A \in \mathbb{R}^{n\times n}$, we want to find either an eigenvalue or all eigenvalues.

Suppose we have $\lambda_1, \lambda_2, \ldots, \lambda_n$, where $|\lambda_1| \geq |\lambda_2| \geq \ldots \geq |\lambda_n|$

Consider the special case where $A$ has a linearly independent set of eigenvectors (which is true when $A$ is symmetric).

Let our eigenvectors be $\vec{x}_1, \vec{x}_2, \ldots, \vec{x}_n$ with $\vec{x}_i$ corresponding to $\lambda_i$.

So $A\vec{x}_i = \lambda_i\vec{x}_i$, with the iterative method, we can start with initial guess $\vec{x}^{\,(0)}$

$$
\begin{aligned}
\vec{x}^{\,(0)} &= \alpha_1\vec{x}_1 + \alpha_2\vec{x}_2 + \ldots + \alpha_n\vec{x}_n \\
A\vec{x}^{\,(0)} &= A\alpha_1\vec{x}_1 + A\alpha_2\vec{x}_2 + \ldots + A\alpha_n\vec{x}_n \\
A\vec{x}^{\,(0)} &= \alpha_1 A\vec{x}_1 + \alpha_2 A\vec{x}_2 + \ldots + \alpha_n A\vec{x}_n \\
A\vec{x}^{\,(0)} &= \alpha_1\lambda_1\vec{x}_1 + \alpha_2\lambda_2\vec{x}_2 + \ldots + \alpha_n\lambda_n\vec{x}_n \\
A^k\vec{x}^{\,(0)} &= \alpha_1\lambda_1^k\vec{x}_1 + \alpha_2\lambda_2^k\vec{x}_2 + \ldots + \alpha_n\lambda_n^k\vec{x}_n \\
&\approx \alpha_1\lambda_1^k\vec{x}_1 \text{ which is our approximate eigenvector}
\end{aligned}
$$

The convergence speed of this approximation is related to $\left|\frac{\lambda_2}{\lambda_1}\right|^k$ (compared to $\varphi(\mathsf{T}^k)$).

There are two main issues with this approximation.

1. $\lambda_1^k \alpha_1 \vec{x}_1 \to \infty$ or 0 which is not our eigenvector.
2. $\lambda_1^k \alpha_1 \vec{x}_1 \to 0$ if $\alpha_1 = 0$

## 6.2 Power Method

1. Start with initial guess $\vec{x}^{(0)}$.
2. Find component largest in magnitude and replace $\vec{x}^{(0)}$ by $\vec{x}^{(0)}$ divided by largest component. This is called normalization.
3. Perform $A\vec{x}^{(0)}$ and call result $\vec{x}^{(1)}$
4. Normalize $\vec{x}^{(1)}$, this is our approximate eigenvector.
5. Repeat from step 3.

But what about other eigenvalues?

### 6.2.1 Inverse Power Method

1. Start with initial guess $\vec{x}^{(0)}$ and normalize it.
2. Choose a $q$ which will be our initial guess for eigenvalue.
3. Find the LU factorization of $A - qI$, where $LU = A - qI$
4. For each step, $LU\vec{y}^{(k+1)} = \vec{x}^{(k)}$
5. Let $U\vec{y} = \vec{z}$ and solve $L\vec{z} = \vec{x}$
6. Solve $U\vec{y} = \vec{z}$ for $\vec{y}$
7. Normalize $\vec{y}^{(k+1)}$ to get $\vec{x}^{(k+1)}$

$A^{-1}$ has eigenvalues $\frac{1}{\lambda_1}, \frac{1}{\lambda_2}, \ldots, \frac{1}{\lambda_n}$ with $|\frac{1}{\lambda_n}| \geq |\frac{1}{\lambda_{n-1}}| \geq \ldots \geq |\frac{1}{\lambda_1}|$
Power method on $A^{-1}$ gives $\frac{1}{\lambda_n}$

# 7 Root-finding Problems

The root finding problem is defined as $f(x) = 0$.

**Intermediate Value Theorem (IVT)**
If $f(x)$ is continuous in $[a, b]$ and if you choose any $M$ between $f(a)$ and $f(b)$. Then $\exists f(c), c \in [a, b]$ *s.t.* $f(c) = M$

Now consider $M = 0$
If $f$ is continuous in $[a, b]$ and $f(a), f(b)$ have different signs. Then $\exists\ c \in [a, b]$ *s.t.* $f(c) = 0$, where $c$ is a root.

If $a, b$ are close to each other *s.t.* $|a - b| \leq \epsilon$ and $f(a), f(b)$ have different signs. Then for any $c \in [a, b]$, $|c - r| \leq |a - b| \leq \epsilon$

## 7.1 Bisection Method

If we have a sufficiently small interval with the root inside, then any point within the interval is a good approximation of the root. However, if we chose the midpoint of the interval, we can reduce the error bound by half, $|c - r| \leq \frac{b-a}{2}$. The bisection method relies on this to reduce the interval and have our guesses converge on the root itself.

1. Suppose initial interval $[a_0, b_0]$ with $f(a), f(b)$ having different signs.
2. Find the midpoint $c_0 = \frac{a+b}{2}$, this is our zeroth approximation.
3. Find $f(c_0)$ and check its sign.
4. Choose $[a_0, c_0]$ or $[c_0, b_0]$ depending on which has different signs.
5. Rename the new interval $[a_1, b_1]$ and find $c_1$.

Eventually, $c_k \to r$ and $|c_k - r| \to 0$, because

$$|c_k - r| \leq \frac{b_k - a_k}{2} = \frac{1}{2}\frac{b_{k-1} - a_{k-1}}{2} = \frac{b_0 - a_0}{2^{k+1}} \to 0$$

*Note: different signs can be ambiguous, mathematically we can use* $f(a)f(b) \leq 0 \implies$ *different signs.*

$\boxed{ex}$ How many iterations until we get error $\leq 10^{-6}$ with initial bounds $[2, 3]$?

$$\begin{aligned} |c_k - r| &\leq \frac{b_0 - a_0}{2^{k+1}} \\ &\leq \frac{1}{2^{k+1}} \\ \Rightarrow 2^{k+1} &\geq 10^6 \\ k + 1 &\geq \log_2 10^6 \\ k + 1 &\geq 6 \log_2 10 \\ k &\geq 6 \log_2 10 - 1 \approx 18.932 \Rightarrow c_{19} \end{aligned}$$

## 7.2 Newton's Method / Tangent Line Approximation

This method uses the root of the tangent line at an initial guess to approximate the root of the function.

$$\begin{aligned} y &= f'(p_0)(x - p_0) + f(p_0) \text{ where } p_0 \text{ is initial guess} \\ 0 &= f'(p_0)(x - p_0) + f(p_0) \\ x &= p_0 - \frac{f(p_0)}{f'(p_0)} \\ \text{Set } p_1 &= p_0 - \frac{f(p_0)}{f'(p_0)} \\ p_{k+1} &= p_k - \frac{f(p_k)}{f'(p_k)} \end{aligned}$$

There are several cases where Newton's method fails:

1. Initial guess has a tangent slope of 0.
2. The function has a vertical slope at the root. This causes certain guesses to bounce around the root rather than converge.

### 7.2.1 Error & Convergence for Newton's Method

Using the Taylor Series between $r$ and $p_k$:

$$e_{k+1} = \frac{f''(e_k)}{2f'(p_k)} e_k^2$$

**Def:** If $e_k \to 0$ and

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|} \to c \qquad \text{This is linear convergence.}$$

$$\lim_{k\to\infty} \frac{|e_{k+1}|}{|e_k|^2} \to c \qquad \text{This is quadratic convergence.}$$

**Thm**

If $f$ is twice continuously differentiable around root $r$ and $f'(r) \neq 0$ and $p_0$ sufficiently close to $r$. Then Newton's method converges for initial guess $p_0$ with quadratic convergence.

## 7.3 Secant Method

$$p_{n+1} = p_n - \frac{f(p_n)(p_n - p_{n-1})}{f(p_n) - f(p_{n-1})}$$

In terms of function evaluations, the secant method requires 1 function evaluation per iteration, similar to Bisection method and dissimilar to Newton's method's 2 function evaluation per iteration. Its drawbacks are that it requires 2 initial guesses $p_0$ and $p_1$.

### 7.3.1 Error

$$\begin{aligned} e_{n+1} &\approx K e_n^{\frac{1+\sqrt{5}}{2}} \\ &\approx K e_n^{1.618} \end{aligned}$$

Compare this to $e_n^2$ of Newton's method. This is called super-linear convergence.

### 7.3.2 Speed

For 2 function evaluations,

$$\begin{aligned} \text{Bisection} \quad & e_{n+2} \approx \frac{1}{4} e_n \\ \text{Newton} \quad & e_{n+1} \approx C e_n^2 \\ \text{Secant} \quad & e_{n+2} \approx K e_n^{2.618} \end{aligned}$$

# 8 Interpolation

## 8.1 Lagrange Interpolating Polynomial

Given the data points $(x_0, y_0), (x_1, y_1), ..., (x_n, y_n)$ where $x_k$ is a node and $(x_k, y_k)$ is a data point. How do we find a polynomial that interpolates them?

Since there are infinitely many polynomials that can fit a set of data points, we want to choose the "simplest," or the lowest degree polynomial possible. Specifically, since there are $n+1$ data points, we want a polynomial of degree $\leq n$.

### 8.1.1 Existence

Idea: find a degree $\leq n$ polynomial that satisfies

| node | $x_0$ | ... | $x_k$ | ... | $x_n$ |
|---|---|---|---|---|---|
| value | 0 | ... | 1 | ... | 0 |

We will call this polynomial $L_{n,k}$

$$\begin{aligned} \text{Then we have } p(x) &= y_0 L_{n,0}(x) + y_1 L_{n,1}(x) + ... + y_n L_{n,n}(x) \\ p(x_1) &= 0 + y_1 + 0 + ... + 0 \\ p(x_k) &= y_k \end{aligned}$$

$$L_{n,k}(x) = \prod_{j=0, j\neq k}^{n} \frac{(x - x_j)}{(x_k - x_j)}$$

ex Find an interpolating polynomial for the following data points.

| $x$ | $-1$ | 2 | 1 |
|---|---|---|---|
| $y$ | 3 | 1 | $-4$ |

Since there are $n + 1 = 3$ data points, we want a polynomial of degree $n = 2$ or less. So a parabola, or preferably, a line.

$$\begin{aligned} \text{Recall that } p(x) &= y_0 L_{n,0}(x) + y_1 L_{n,1}(x) + ... + y_n L_{n,n}(x) \\ &= 3\frac{(x-2)(x-1)}{(-1-2)(-1-1)} + 1\frac{(x+1)(x-1)}{(2+1)(2-1)} - 4\frac{(x+1)(x-2)}{(1+1)(1-2)} \end{aligned}$$

Here this $p(x)$ is said to be in Lagrange form.

### 8.1.2 Uniqueness

**Fundamental Theorem of Algebra (FTA)**

A non-constant polynomial of degree $n$ has $n$ roots in the complex plane. Then if $p(x)$ is a polynomial of degree $\leq n$ and suppose we find $n+1$ distinct roots then $p$ must be a constant.

Let $p, q$ be polynomials of deg $\leq n$ that interpolate the data points with distinct nodes $x_0, x_1, ..., x_n$. Look at their difference $r = p - q$, which is a polynomial of deg $\leq n$.

$$r(x_i) = p(x_i) - q(x_i) = y_i - y_i = 0$$

at each node $x_i$ for $0 \leq i \leq n$. Therefore $r$ has $n+1$ roots. Yet its degree is $\leq n$. Therefore, $r$ is a constant polynomial. Since we evaluated it at $x_i$ which resulted in zero, it must be zero, guaranteeing uniqueness.

### 8.1.3 Error

**Thm**

Let $(x_0, f(x_0)), (x_1, f(x_1)), ..., (x_n, f(x_n))$ be $n+1$ data points with distinct nodes. Suppose that $f \in \mathbb{C}^{n+1}\ [a, b]$ is $n+1$ times differentiable where $x_i \in [a, b]\ \forall\ i$. Let $p$ be the interpolating polynomial. Then

$$f(x) = p(x) + \frac{f^{(n+1)}(\xi_x)}{(n+1)!}(x - x_0)(x - x_1)...(x - x_n)$$

where $\xi_x \in [a, b]$ or $\left(\min_{0 \leq i \leq n} x_i, \max_{0 \leq i \leq n} x_i\right)$.

When our chosen $x$ is far from the nodes, then $|x - x_i|$ becomes large and our error also becomes large. The same happens when derivatives are large. To counter this effect and have small error, we can find $\max |f^{(n+1)}(x)|$ and vary $x, x_0, x_1, ..., x_n$ to counter it with $\prod_{j=0}^{n}(x - x_j)$. We can also use less data points than available, creating piecewise linear or quadratic functions.

## 8.2 Piecewise Interpolating Polynomials

### 8.2.1 Piecewise Linear Interpolating Polynomials

ex Suppose the following data points. What is the value of the piecewise linear polynomial at $x = 0.32$.

| x | 0 | 0.1 | 0.2 | 0.3 | 0.4 | 0.5 |
|---|---|---|---|---|---|---|
| y | 2.1 | 2.4 | 2.7 | 3 | 1.7 | 1.2 |

1. Find correct interval $(0.3, 3), (0.4, 1.7)$
2. Find interpolating polynomial

$$p(x) = 3\left(\frac{x - 0.4}{0.3 - 0.4}\right) + 1.7\left(\frac{x - 0.3}{.4 - .3}\right)$$
$$\text{or } p(x) = \frac{1.7 - 3}{0.4 - 0.3}(x - 0.3) + 3$$

3. Evaluate interpolating polynomial

$$\begin{aligned} p(0.32) &= \frac{3(0.08)}{0.1} + \frac{1.7(0.02)}{0.1} \\ &= 2.4 + 0.34 \\ &= 2.74 \end{aligned}$$

### 8.2.2 Smoothness

Piecewise linear interpolating polynomials are very jagged, we wish to find something smoother. This comes from degrees of freedom, or unknowns, which is the coefficients of the pieces of the polynomials.

Consider a polynomial with degree $\leq k$. There are $(k+1)n$ total number of subintervals, and for piecewise quadratics, $2n + (n-1)$ conditions, where $2n$ is the condition for interpolation and continuity, and $n - 1$ is the condition for a continuous $1^{st}$ derivative.

In general, for existence of solutions, we want the number of unknowns $\geq$ the number of equations. and so we want $(k+1)n \geq 2n + (n-1) = 3n - 1$. In this case we choose $k = 2$ which means piecewise quadratics, giving $3n$ unknowns

on $3n-1$ equations. In general, we can create a boundary equation which is a condition at $x_0$ or $x_n$ to get $3n$ equations.

We can impose an additional condition so that our interpolating polynomial becomes even smoother. This condition is that second derivatives must match on both sides of junction points.

$$p_i''(x_1) = p_{i+1}''(x_1) \text{ for } i = 0, 1, 2, ..., n-2$$
$$\text{conditions: } 2n + (n-1) + (n-1) = 4n - 2$$

So we have $(k+1)n \geq 4n-2$. We can set $k = 3$ and create 2 more conditions to match the number of unknowns. Since $k = 3$ this gives us a piecewise cubic function for a cubic spline.

Here are the possible conditions we can create:

1. Clamped boundary conditions. This can be thought of as prescribed angles for the endpoints of the spline.

$$p_0'(x_0) = \text{given}$$
$$p_{n-1}'(x_n) = \text{given}$$

2. Free/natural boundary conditions. This can be thought of as an infinite rod held at the end points, bent to pass through all data points.

$$p_0''(x_0) = 0$$
$$p_{n-1}''(x_n) = 0$$

3. Not-a-knot boundary conditions.

## 8.3 Least-squares Polynomial

$$E(a, b) = \sum_{i=1}^{m} (y_i - (ax_i + b))^2$$

We want an $a, b$ that minimizes this error. We can do this by finding the critical points. We want $\frac{\partial E}{\partial a} = 0$ and $\frac{\partial E}{\partial b} = 0$.

This results in the following matrix form of the normal equations

$$\begin{bmatrix} \sum_{i=1}^{m} 1 & \sum_{i=1}^{m} x_i \\ \sum_{i=1}^{m} x_i & \sum_{i=1}^{m} x_i^2 \end{bmatrix} \begin{bmatrix} b \\ a \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^{m} y_i \\ \sum_{i=1}^{m} x_i y_i \end{bmatrix}$$

For a best fitting polynomial of degree $\leq n$

$$E(a_0, a_1, ..., a_n) = \sum_{i=1}^{m} (y_i - (a_0 + a_1 x_i + a_2 x_i^2 + ... + a_n x_i^n))^2$$

# 9 Numerical Differentiation

## 9.1 Differencing

Forward differencing (derivative from the right) is defined as

$$\frac{f(x+h) - f(x)}{h}$$

Backward differencing (derivative from the left) is defined as

$$\frac{f(x) - f(x-h)}{h}$$

Central differencing is defined as

$$\frac{f(x+h) - f(x-h)}{h}$$

Using Taylor series, we can find the error associated with each of these methods of finding the derivative. This yields $\mathcal{O}(h)$ for both forward and backward differencing and $\mathcal{O}(h^2)$ for central differencing. To optimize error, we want to use forward differencing on the first node, backward differencing on the last node, and central differencing for all the ones in between.

## 9.2 Method of Undetermined Coefficients

Method of Undetermined Coefficients yield the following formula for nodes $x_0, x_1, x_2$.

$$\frac{-3f(x_0) + 4f(x_1) - f(x_2)}{2h}$$

# 10 Numerical Integration

## 10.1 Piecewise Constant Interpolation

### 10.1.1 Composite Left Endpoint Rule

$$\int_a^b f(x)dx \approx h(f(x_0) + f(x_1) + ... + f(x_{n-1}))$$

### 10.1.2 Composite Right Endpoint Rule

$$\int_a^b f(x)dx \approx h(f(x_1) + f(x_2) + ... + f(x_n))$$

### 10.1.3 Composite Midpoint Rule

$$\int_a^b f(x)dx \approx 2h(f(x_1) + f(x_3) + ... + f(x_{n-1}))$$

Only possible if $n$ is even. This is also as accurate as trapezoid rule, which is $\mathcal{O}(h^2)$.

## 10.2 Piecewise Linear Interpolation

$$\int_a^b f(x)dx \approx \frac{h}{2}(f(x_0) + 2f(x_1) + ... + 2f(x_{n-1}) + f(x_n))$$

This is called the composite trapezoid rule. The coefficients of 2 are due to double contributions at junction points. Its error is $\mathcal{O}(h^2)$.

## 10.3 Piecewise Quadratic Interpolation

$$\int_a^b f(x)dx \approx \frac{h}{3}(f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + ... + 4f(x_{n-1}) + f(x_n))$$

This is called the composite Simpson's rule. Only possible if $n$ is odd. Its error is $\mathcal{O}(h^4)$.